

RAPID: Robot Agentic Programming from Demonstrations for Nonprehensile Manipulation

Abstract—This is the abstract.

I. INTRODUCTION

Recently, coding agents [1], [2] have shown remarkable capabilities in developing complex software systems, sparking growing interest in solving robotic manipulation tasks through programming [3]–[8]. Consider how human engineers would develop a robot program to solve the task in Figure 1 before the era of coding agents: given the (a) *task specification* (pick up the cookie box), they would write a program that composes the available (b) *manipulation primitives* (push the box to a fixture, flip it against the fixture, then grasp and lift it), and test the program in (c) *interactive environments* to verify its correctness and iteratively debug it. This process mirrors the core strength of coding agents: not one-shot code generation, but the iterative cycle of generating, executing, verifying, and revising a program, commonly referred to as the agentic coding loop [9], [10].

For the agentic coding loop to work in robotic manipulation, it requires three ingredients: task specification, manipulation primitives, and interactive environments for verification. Existing robotic coding-agent systems typically assume these ingredients are provided through human-engineered primitives, explicit task objectives, and resettable environments with reward or success feedback. This raises a fundamental question: **how can a coding agent close the loop when these ingredients are not provided a priori?**

We explore *demonstrations* as a programming interface for supplying the task- and motion-level information needed by the agentic coding loop. A demonstration directly shows a feasible way to accomplish a task and, together with a language description, conveys the intended outcome. Rather than merely reproducing the demonstrated trajectory, we aim to recover an executable and reusable manipulation program. Demonstrations, however, do not provide the interactive environment needed to verify and iteratively refine that program, posing an additional challenge for verification.

Motivated by this perspective, we introduce **Robot Agentic Programming from Demonstrations (RAPID)**, a framework for constructing and iteratively refining robot programs from a single human demonstration with a coding agent. RAPID infers the task specification, constructs the required manipulation primitives and strategy, and reconstructs the demonstrated scene in simulation for iterative program verification and refinement. To improve generalization, RAPID additionally generates scene variants for further verification. In this way, RAPID automatically establishes the three

Teaser Figure Placeholder

Fig. 1: teaser

ingredients required to close the agentic coding loop for robotic manipulation.

In this work, we apply RAPID to *nonprehensile manipulation*, where the need for such a framework is especially acute. Many prehensile tasks can be composed from a compact and well-established set of reusable primitives [11], including *pick*, *move in free space*, and *place*. Nonprehensile manipulation, by contrast, spans diverse and geometry-dependent contact interactions, including pushing [12], [13], pivoting [14], [15], toppling [16], flipping [17], and sliding [18]. Capturing this diversity with a fixed primitive library is difficult, making manual primitive design a significant engineering burden. This makes nonprehensile manipulation a natural testbed for asking whether a coding agent can construct reusable manipulation primitives from a single demonstration rather than rely on human engineering.

A central challenge is *generalization* beyond a single demonstrated scene. Programs that directly encode poses, distances, or waypoints may reproduce the demonstrated behavior but can fail when the scene changes. RAPID instead captures the behavior in terms of object-centric relations and task-relevant effects. Specifically, RAPID represents primitives as relational trajectory-optimization programs that realize object-level motion effects, such as pushing an object by a displacement or flipping it by an angle, while resolving scene-dependent geometry at execution time. At the task level, primitives are connected into a strategy through relational constraints that determine how the outcome of one phase specifies the next. For example, rather than replaying a fixed pushing distance, the program can compute how far an object should be pushed from its current pose to establish contact with a fixture. By expressing primitives and their composition relationally, RAPID preserves the invariant structure of the demonstrated strategy while adapting to changes in object

pose, size, geometry, and physical properties.

The remaining challenge is *verification*: the coding agent needs an interactive environment in which candidate programs can be executed, evaluated, and revised. RAPID reconstructs such an environment in simulation from the demonstration and augments it with scene variants that perturb object geometry, pose, appearance, and physical properties. Agent-generated filters retain only task-consistent and physically feasible variants. The coding agent then executes and iteratively revises the program across this verification set. This simulation-based verification closes the agentic loop while reducing overfitting to the demonstrated scene.

We evaluate RAPID on 8 contact-rich nonprehensile manipulation tasks, focusing on whether a program constructed from one demonstrated instance can generalize to substantial changes in object pose, geometry, physical properties, and scene configuration. We additionally evaluate the frozen programs on physical robots and conduct ablations to assess the contributions of the relational program representation, scene variation, and iterative verification.

Our contributions are threefold:

- We introduce RAPID, a framework that closes the agentic coding loop for robotic manipulation by automatically constructing (a) *task specifications*, (b) *manipulation primitives*, and (c) *interactive verification environments*, all from a single human demonstration.
- We introduce *relational manipulation programs*, which represent manipulation primitives as parameterized trajectory-optimization programs and compose them through relational constraints, enabling reusable strategies that adapt across scenes.
- We demonstrate that RAPID can construct reusable programs from a single human demonstration across diverse contact-rich nonprehensile manipulation tasks, generalize to substantial scene variations, and be deployed in the real world.

II. PROBLEM STATEMENT

We consider programming a reusable *nonprehensile manipulation* strategy from a single visual human demonstration D and a language instruction l . The demonstration records a human performing the task in an example scene. The resulting program should solve a family of task variants $\mathcal{E}$ that share the same task objective but may differ in object appearance, geometry, pose, physical properties, and scene configuration.

We assume that no manipulation primitives, explicit success criterion, or instantiated simulation environment are provided a priori. Given only (D, l) , the agent must first construct a set of executable manipulation primitives

$$\mathcal{P} = \{P_1, \dots, P_N\},$$

and then compose these primitives into a strategy Π . The resulting program should capture the manipulation structure underlying the demonstration and generalize beyond the demonstrated instance:

$$(D, l) \longrightarrow (\mathcal{P}, \Pi), \quad \Pi(E; \mathcal{P}) \text{ succeeds for } E \in \mathcal{E}.$$

The goal is to recover reusable primitives and a strategy that can adapt to unseen task variants.

III. RELATIONAL MANIPULATION PROGRAMS

As shown in Figure 2, RAPID uses agentic programming to construct a reusable robot manipulation strategy Π given a single visual human demonstration D and its corresponding language instruction l . It first infers the task specification from (D, l) and generates an initial object-centric, relational program for accomplishing the task. RAPID then reconstructs an interactive simulation environment from D , in which it iteratively executes, verifies, and revises the program. To improve reusability and generalization, RAPID also proposes several scene variants, verifies the program across them, and uses the failures to guide revision. At deployment, the strategy is frozen and can be instantiated in a novel scene through lightweight semantic binding.

In this section, we introduce the object-centric and relational program representation of primitives, how primitives are composed to form a strategy, and how a strategy is instantiated in novel scenes. We then describe how RAPID constructs and verifies these programs from a single human demonstration in Sec. IV.

A. Primitives as Relational Trajectory-Optimization Programs

A primitive is a reusable behavior for realizing an object-level motion effect, such as pushing for a displacement or flipping for an angle. We represent a primitive as a relational trajectory-optimization program

$$P_i = (G_i, C_i, \Psi_i, \rho_i, J_i) \in \mathcal{P}.$$

Here, G_i is a semantic, object-centric description of the intended effect; C_i is the relational interface; Ψ_i is the verifiable success condition; ρ_i is the geometry resolver; and J_i is the optimization cost. Together, these components specify how the primitive is instantiated, optimized, and verified in a specific scene. A primitive is therefore neither a fixed trajectory nor a black-box policy. As a concrete example, Table I illustrates each component for flipping an object against a wall. We now explain the components in detail.

Relational interface. We write the primitive interface as

$$C_i = (\mathbf{r}_i, \Gamma_i, \mathbf{q}_i).$$

The declarations $\mathbf{r}_i$ and $\mathbf{q}_i$ specify the semantic scene roles and typed effect-parameter slots required by P_i , while Γ_i specifies

TABLE I: Relational program for flipping an object against a wall.

Component	Content
Semantic goal G_i	Flip the target against the vertical fixture for a specific angle.
Relational contract C_i	Semantic roles $\mathbf{r}_i$: target, support plane, fixture, and pusher. Relations Γ_i : the target rests on the support plane, and the fixture exposes a reachable vertical face. Effect parameter $\mathbf{q}_i$: flipping angle.
Success predicate Ψ_i	Check that the achieved rotation matches the flipping angle and that the target remains supported, near the fixture, and settled.
Geometry resolver ρ_i	At each rollout state, derive the support plane, fixture face, pivot axis, target faces, dimensions, and clearances.
Final cost J_i^f	Penalize terminal rotation error relative to the flipping angle, support and fixture gaps, and terminal velocity.
Progress cost J_i^p	Encourage rotation and pusher proximity while discouraging backtracking and irregular motion.

Method Figure Placeholder

Fig. 2: Overview of RAPID.

the relations among the roles. An invocation supplies runtime arguments (b_i, g_i) satisfying

$$b_i \in \text{Bind}_E(\mathbf{r}_i, \Gamma_i), \quad g_i \in \text{Val}(\mathbf{q}_i).$$

Here, $\text{Bind}_E(\mathbf{r}_i, \Gamma_i)$ denotes the type-compatible assignments of scene entities from the environment E to $\mathbf{r}_i$ that satisfy Γ_i , and $\text{Val}(\mathbf{q}_i)$ denotes the values matching the types declared by $\mathbf{q}_i$. Thus, b_i instantiates the role declarations and g_i instantiates the effect-parameter declarations. In the flipping example (Table I), b_i binds the target, support plane, fixture, and pusher to scene entities, while g_i supplies the requested angle. Scene-dependent geometry remains internal to the primitive and is resolved by ρ_i at execution time.

Geometry resolver. Let $\mathcal{X}(E)$ be the state space of environment E . At each state $x_t \in \mathcal{X}(E)$ along a rollout, the geometry resolver computes

$$\phi_{i,t} = \rho_i(E, x_t, b_i), \quad t = 0, \dots, T,$$

where b_i assigns the primitive’s semantic roles to concrete entities in E , and $\phi_{i,t}$ collects the corresponding geometric features at time t .

Success predicate. For a terminal state $x_T \in \mathcal{X}(E)$, the predicate $\Psi_i(E, x_T; g_i, \phi_{i,T}) \in \{0, 1\}$ provides an executable interpretation of G_i . It evaluates the achieved object relations and requested effect, yielding an unambiguous signal for verification.

Optimization cost. For a simulated trajectory $\tau = (x_0, \dots, x_T)$ in environment E , where $x_t \in \mathcal{X}(E)$, the primitive cost is

$$J_i(E, \tau; g_i, \phi_{i,0:T}) = J_i^f(E, x_T; g_i, \phi_{i,T}) + \lambda_i J_i^p(E, \tau; g_i, \phi_{i,0:T}).$$

Here, J_i^f is the final cost, which measures how far x_T is from the effect encoded by G_i ; J_i^p is the progress cost, which may depend on the full trajectory and guides the optimizer through contact-rich interaction heuristics; and $\lambda_i \geq 0$ is a constant. The coding agent proposes the heuristics by reasoning about the visual human demonstration.

During primitive construction, the agent also proposes an initialization heuristic based on the resolved geometry and requested effect. The heuristic provides a plausible interaction trajectory from which to begin the search. Let z parameterize a candidate robot motion, let $\tau(z) = \text{Rollout}_E(x_0, z)$ denote its simulated state trajectory from x_0 in environment E , and let $\phi_{i,0:T}(z)$ be the feature sequence resolved along that trajectory.

The executable trajectory is obtained as

$$z_i^* = \arg \min_z J_i(E, \tau(z); g_i, \phi_{i,0:T}(z)).$$

We solve this trajectory-optimization problem with the Cross-Entropy Method (CEM) [19], a sampling-based optimizer. The primitive thereby adapts its motion to the object pose, size, geometry, physical properties, and requested effect.

B. Strategies as Primitive Composition by Constraints

Suppose we have a library of primitives $\mathcal{P}$, where each $P_i \in \mathcal{P}$ realizes an object-level motion effect. A strategy Π is a reusable task-level program that composes these primitives to accomplish an overall task. It specifies the sequence of primitives to invoke, how task roles map to each primitive, and how the requested effect of each invocation is determined:

$$\Pi = \left(G_\Pi, C_\Pi, \Psi_\Pi, \langle (P_{i_k}, m_k, c_k) \rangle_{k=1}^K \right),$$

where G_Π describes the overall task goal and Ψ_Π evaluates its final outcome.

Strategy interface. A strategy exposes semantic roles and their required relations,

$$C_\Pi = (\mathbf{r}_\Pi, \Gamma_\Pi).$$

To invoke the strategy, a role binding $b \in \text{Bind}_E(\mathbf{r}_\Pi, \Gamma_\Pi)$ assigns an entity to each strategy role while satisfying Γ_Π . Primitive effect parameters are not exposed through C_Π ; instead, they are derived inside the strategy during execution.

Primitive Composition by Constraints. A strategy consists of K phases, each represented by a tuple (P_{i_k}, m_k, c_k) , where $P_{i_k} \in \mathcal{P}$ is a primitive, m_k is a role map, and c_k is a connecting constraint. For the invoked primitive P_{i_k} , m_k produces a role binding b_k that fills the role declarations $\mathbf{r}_{i_k}$, while c_k produces an effect argument g_k that fills the parameter declarations $\mathbf{q}_{i_k}$. The connecting constraint declaratively specifies how phase k depends on the outcome of phase $k-1$ and is evaluated just in time at the beginning of phase k :

$$\begin{aligned} b_k &= m_k(b) \in \text{Bind}_E(\mathbf{r}_{i_k}, \Gamma_{i_k}), \\ g_k &= c_k(E, s_{k-1}, b) \in \text{Val}(\mathbf{q}_{i_k}), \end{aligned}$$

where $s_{k-1} \in \mathcal{X}(E)$ denotes the environment state after the execution of phase $k-1$. The phase is then executed as

$$s_k = \text{Exec}(P_{i_k}; E, s_{k-1}, g_k, b_k).$$

Here, Exec solves the local trajectory-optimization problem of P_{i_k} and executes the optimized trajectory.

For example, consider a phase that establishes target–fixture contact using a pushing primitive. The role map m_k maps the strategy’s target, support, and tool roles to the corresponding roles exposed by the primitive, producing the primitive binding b_k . The connecting constraint c_k then computes the required displacement from the current target and fixture poses and expresses it in the primitive’s relational input frame, producing the effect argument g_k .

C. Strategy Instantiation in Novel Scenes

To deploy a strategy in a novel scene, we only need to provide a semantic binding. Given a novel scene E^* and instruction I^* , RAPID uses a coding agent to rapidly generate a semantic binding $b^* \in \text{Bind}_{E^*}(\Gamma_\Pi, \Gamma_\Pi)$ that assigns scene entities to the semantic roles exposed by the strategy interface. Since this step only associates the exposed semantic roles with scene entities, it is lightweight and incurs little deployment overhead. At phase k , the strategy derives the primitive binding and effect argument automatically.

Throughout this process, the strategy program Π and the referenced primitives remain frozen. Generalization therefore arises from the relational program representation, together with the adaptation of trajectory optimization to the geometry and physical properties of the new scene.

IV. AGENTIC PROGRAMMING FROM DEMONSTRATION

In Sec. III, we introduced a reusable and generalizable representation of relational manipulation programs that can be efficiently instantiated in novel scenes. We now describe how RAPID employs a coding agent to construct and iteratively refine such programs from a single visual human demonstration. Specifically, RAPID closes the agentic loop with the single demonstration by automatically inferring the task specification, generating programs the manipulation primitives referred by the strategy, and reconstructing the interactive simulation environments used for verification.

A. Reconstructing Interactive Environments

Interactive environment. To verify a manipulation program, RAPID reconstructs an interactive simulation environment from the initial RGB-D frame of the demonstration. We first use a vision-language model (VLM) to identify the objects in the scene, assign them with semantic labels, and estimate their physical properties. Then, for each semantic name, we utilize SAM 3 [20] to get a segmentation mask. The RGB-D frame and the mask are then fed into SAM 3D [21] to reconstruct a complete mesh. Finally, we use FoundationPose [22] to register each mesh to the RGB-D frame, and assemble the reconstructed objects and robot model in MuJoCo [23]. The resulting environment preserves the scene entities, geometry, and spatial relations needed to execute the candidate program and evaluate its success predicates.

Feasible scene variants. Verifying a program only in the reconstructed environment from a single demonstration may

leave accidental assumptions about the demonstration undetected. To improve reusability and generalizability, RAPID uses a task-agnostic variation sampler to generate scene variants. This sampler will randomize object appearances, sizes, poses, as well as physical parameters such as mass, friction, and elasticity. However, the randomly sampled scenes might not be feasible for a certain primitive or strategy. Therefore, we will let the coding agent to write a filter to leave out those infeasible variants based on its understanding of the primitive or the strategy. A variant is accepted only if it preserves the declared roles and relations, retains the task meaning, and remains physically feasible.

B. Primitive Construction

To construct the primitives, we first need to decompose the demonstration of the overall tasks into a sequence of interaction phases. Given the full demonstration D and its natural language description l , RAPID calls a subagent to partition D into an ordered sequence of interaction segments $(D_1, \dots, D_K)$ and assigns a local description l_i to each D_i . For each (D_i, l_i) , if the coding agent recognizes that the behavior is covered by an existing primitive in the primitive library $\mathcal{P}$, we can simply retrieve this primitive. Otherwise, the coding agent will infer a primitive specification consisting of a semantic goal G_i and a success predicate Ψ_i from (D_i, l_i) . It further reasons about the contact interactions that produces the demonstrated effect, the relational structure that should remain invariant across executions, and the quantities that might vary between invocations. For example, for the flipping primitive in Table I, the demonstrated rotation supplies the flipping-angle argument, while the required relations among the target, support, fixture, and pusher define the reusable interaction structure.

Given the inferred primitive specification and the analysis about physical interaction structure, the coding agent will generate the relational primitive contract C_i , geometry resolver ρ_i , and optimization cost J_i introduced in Sec. III-A. RAPID then iteratively verifies and revises the resulting primitive in the reconstructed interactive environment $\hat{E}_i$ as well as in automatically generated scene variants. The complete process is illustrated in Algorithm 1: RAPID reconstructs the segment-level environment $\hat{E}_i$, proposes a primitive program P_i , generates feasible scene variants, and revises the program until the success predicate Ψ_i is satisfied across the verification set $\mathcal{V}_i$. Finally, the primitive is added to the library $\mathcal{P}$.

C. Strategy Composition

After constructing the required primitives, RAPID composes them into a task-level strategy. Given the full demonstration D and instruction l , the coding agent infers a strategy specification consisting of the overall task goal G_Π and final success predicate Ψ_Π . The sequence of the interaction segments $(D_1, \dots, D_K)$ determines the ordering of the primitive composition. The agent then constructs the strategy interface C_Π and, for each phase k , selects a verified primitive P_{i_k} , defines the role map m_k , and specifies the

Algorithm 1: RAPID

Input: Human demonstration D , instruction l
Output: Verified primitives $\mathcal{P}$ and strategy Π
 $\{(D_i, l_i)\}_{i=1}^K \leftarrow \text{SEGMENT}(D, l)$
 $\mathcal{P} \leftarrow \emptyset$
// *Primitive construction*
for $i = 1, \dots, K$ **do**
 $\hat{E}_i \leftarrow \text{RECONSTRUCT}(D_i)$
 $(G_i, \Psi_i) \leftarrow \text{INFEROBJECTIVE}(D_i, l_i)$
 $g_i \leftarrow \text{INFEREFFECT}(D_i, G_i)$
 $P_i \leftarrow \text{PROPOSEPRIMITIVE}(D_i, G_i, \Psi_i, \hat{E}_i)$
 $\mathcal{V}_i \leftarrow \{\hat{E}_i\} \cup \text{VARIANTS}(\hat{E}_i, P_i)$
 $P_i \leftarrow \text{VERIFYREVISE}(P_i, \mathcal{V}_i, (g_i, l_i))$
 $\mathcal{P} \leftarrow \mathcal{P} \cup \{\text{FREEZE}(P_i)\}$
// *Strategy composition*
 $\hat{E}_0 \leftarrow \text{RECONSTRUCT}(D)$
 $(G_\Pi, \Psi_\Pi) \leftarrow \text{INFEROBJECTIVE}(D, l)$
 $\Pi \leftarrow \text{COMPOSE}(D, \mathcal{P}, G_\Pi, \Psi_\Pi, \hat{E}_0)$
 $\mathcal{V}_\Pi \leftarrow \{\hat{E}_0\} \cup \text{VARIANTS}(\hat{E}_0, \Pi)$
 $\Pi \leftarrow \text{VERIFYREVISE}(\Pi, \mathcal{V}_\Pi, l)$
return $\mathcal{P}, \text{FREEZE}(\Pi)$
// *Shared verification and revision*
function $\text{VERIFYREVISE}(Q, \mathcal{V}, a)$
 repeat
 $\mathcal{R} \leftarrow \text{EXECUTEEVALUATE}(Q, \mathcal{V}, a)$
 if $\neg \text{ALLSUCCESSFUL}(\mathcal{R})$ **then**
 $Q \leftarrow \text{DIAGNOSEREVISE}(Q, \mathcal{R})$
 until $\text{ALLSUCCESSFUL}(\mathcal{R})$
 return Q

connecting constraint c_k . Together, these components form an initial version of the strategy Π .

RAPID then iteratively verifies and revises the strategy in the reconstructed interactive environment $\hat{E}_0$ and its feasible scene variants. As illustrated by Algorithm 1, during each execution, RAPID binds the semantic roles, invokes the selected primitives sequentially, evaluates each connecting constraint just in time, and checks the final outcome with Ψ_Π . The failure signals are returned to the coding agent, and the coding agent will revise the strategy composition accordingly while keeping the verified primitives fixed, until Ψ_Π is satisfied across $\mathcal{V}_\Pi$.

V. EXPERIMENTS

VI. RELATED WORK

Agentic Programming for Robotics.
Learning from Few Demonstrations.
Nonprehensile Manipulation.

VII. CONCLUSION

APPENDIX

REFERENCES

- [1] OpenAI, “Codex,” 2025. <https://openai.com/codex/>.
- [2] Anthropic, “Claude Code,” 2025. <https://claude.com/product/claude-code>.
- [3] K. Chen, S. Xie, L. Fu, J. Yu, W. Pacini, S. Bajamahal, H. Kim, J. Drake, D. Kim, H. Xue, *et al.*, “GaP: A Graph-as-Policy Multi-Agent Self-Learning Harness For Variational Automation Tasks,” *arXiv:2607.05369*, 2026.
- [4] W. Xiao, J. Xie, T. Zhang, H. Lin, L. Fu, H. Xue, J. Lu, Y. Yang, C. Dai, Z. Wang, *et al.*, “ENPIRE: Agentic Robot Policy Self-Improvement in the Real World,” *arXiv:2606.19980*, 2026.
- [5] R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, *et al.*, “ASPIRE: Agentic Skills Discovery for Robotics,” *arXiv:2607.00272*, 2026.
- [6] L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, D. Niu, F.-F. Li, *et al.*, “CaP-X: A framework for benchmarking and improving coding agents for robot manipulation,” *arXiv:2603.22435*, 2026.
- [7] S. Berman, M. Ilie, J. Deng, and D. Freeman, “Claude Plays Robotics,” 2026. <https://www.anthropic.com/research/claude-plays-robotics>.
- [8] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as Policies: Language Model Programs for Embodied Control,” in *ICRA*, 2023.
- [9] A. Ng, “Three Key Loops for Building Great Software,” 2026. <https://www.deeplearning.ai/the-batch/three-key-loops-for-building-great-software>.
- [10] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, “AgentCoder: Multi-Agent-Based Code Generation with Iterative Testing and Optimisation,” *arXiv:2312.13010*, 2023.
- [11] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated Task and Motion Planning,” *Annual review of control, robotics, and autonomous systems*, vol. 4, pp. 265–293, 2021.
- [12] M. T. Mason, “Mechanics and Planning of Manipulator Pushing Operations,” *IJRR*, vol. 5, no. 3, pp. 53–71, 1986.
- [13] M. T. Mason, *Mechanics of Robotic Manipulation*. MIT press, 2001.
- [14] A. Holladay, R. Paolini, and M. T. Mason, “A General Framework for Open-Loop Pivoting,” in *ICRA*, 2015.
- [15] Y. Aiyama, M. Inaba, and H. Inoue, “Pivoting: A New Method of Grasplless Manipulation of Object by Robot Fingers,” in *IROS*, 1993.
- [16] X. Cheng, E. Huang, Y. Hou, and M. T. Mason, “Contact Mode Guided Sampling-Based Planning for Quasistatic Dexterous Manipulation in 2D,” in *ICRA*, 2021.
- [17] L. U. Odhner, R. R. Ma, and A. M. Dollar, “Open-Loop Precision Grasping with Underactuated Hands Inspired by a Human Manipulation Strategy,” *T-ASE*, vol. 10, no. 3, pp. 625–633, 2013.
- [18] X. Cheng, E. Huang, Y. Hou, and M. T. Mason, “Contact Mode Guided Motion Planning for Quasidynamic Dexterous Manipulation in 3D,” in *ICRA*, 2022.
- [19] P.-T. de Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein, “A Tutorial on the Cross-Entropy Method,” *Annals of Operations Research*, vol. 134, no. 1, pp. 19–67, 2005.
- [20] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris Coll-Vinent, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, *et al.*, “SAM 3: Segment Anything with Concepts,” in *ICLR*, 2026.
- [21] X. Chen, F.-J. Chu, P. Gleize, K. J. Liang, A. Sax, H. Tang, W. Wang, M. Guo, T. Hardin, X. Li, *et al.*, “SAM 3D: 3Dfy Anything in Images,” in *CVPR*, 2026.
- [22] B. Wen, W. Yang, J. Kautz, and S. Birchfield, “FoundationPose: Unified 6D Pose Estimation and Tracking of Novel Objects,” in *CVPR*, 2024.
- [23] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A Physics Engine for Model-Based Control,” in *IROS*, 2012.

TABLE II: Experiment Results

Task	Skeleton	$\pi_{0.5}$	RAPID w/o Prim.	RAPID w/o Optim.	RAPID w/o Goal	RAPID w/o Rel.	RAPID
Store Box	push→push	–	–	–	–	–	–
	press→flip→push	–	–	–	–	–	–
	rotate→push→push	–	–	–	–	–	–
Grasp Box	push→flip→grasp	–	–	–	–	–	–
	push→flip→press→grasp	–	–	–	–	–	–
	press→push→flip→grasp	–	–	–	–	–	–
Pick up Book	tilt→grasp	–	–	–	–	–	–
	push→grasp	–	–	–	–	–	–

TABLE III: Experiment Results in the Real World

Task	Skeleton	RAPID
Store Box	push→push	–
	press→flip→push	–
	rotate→push→push	–
Grasp Box	push→flip→grasp	–
	push→flip→press→grasp	–
	press→push→flip→grasp	–
Pick up Book	tilt→grasp	–
	push→grasp	–